



Lending Decision Platform

Product Build on Google Cloud Platform

Agentic credit decisioning – with explainability, fairness and compliance built in.

AI/ML Engineering | Cloud Architecture | MLOps



Lending Decision Platform

Business context

Manual underwriting was slow, inconsistent between officers, and hard to audit. We needed automated credit decisioning that is fast, consistent, explainable, and compliant with responsible-lending obligations.

The shape of the solution

A calibrated and gradient boosted ML lending-decision model at the core, wrapped in four specialised AI agents (application intake, KYC/AML, financial analysis, security audit), grounded in current lending policy via RAG, with a human in the loop on high-value / low-confidence cases — all on Google Cloud.

Agenda



1. Personal Contributions

What I built vs. team responsibilities



2. Technology Stack & Trade-offs

GCP services, tools, and key decisions



3. Solution Architecture

End-to-end platform architecture diagram



4. ML Model Training Pipeline

Training workflow with GCP cloud services



5. AI Agent & RAG Pipeline

Retrieval-augmented generation architecture



6. MLOps & Release Management

Fast dev to production, monitoring & metrics



7. Failure Handling & Resilience

Agent hallucination, drift, system failures



1. Personal Contributions

What I personally built vs. team deliverables

My Contributions vs. Team Responsibilities

My Specific Contributions

ML Pipeline Architecture

Designed & built end-to-end ML training pipeline on Vertex AI with feature engineering, model registry, and automated retraining triggers.

AI Agent & RAG System

Built LangChain-based agents with tool orchestration, RAG pipeline using Vertex AI Search, vector embeddings, and policy document retrieval.

Credit Scoring Models

Developed gradient-boosted & deep learning models (XGBoost, TensorFlow) for credit risk, PD/LGD estimation, and affordability checks.

MLOps & Monitoring

Implemented MLflow experiment tracking, model versioning, A/B testing framework, and drift detection using Evidently AI.

Platform Infrastructure (DevOps)

GKE cluster setup, Terraform IaC, networking (VPC, Cloud NAT), IAM policies, and CI/CD pipeline scaffolding.

Data Engineering

Argo Workflow Data Extraction pipelines, data quality checks with Great Expectations.

My Contributions vs. Team Responsibilities

Team Handled (Involved into Collaboration Work with Cross Team)

Data Engineering

BigQuery data warehouse, Dataflow ETL pipelines, CDC from source systems, data quality checks with Great Expectations.

Frontend & API Layer

React dashboard for loan officers, REST/gRPC APIs (Cloud Run), authentication (Firebase Auth), and notification services.

Compliance & Security

Data masking, encryption at rest/transit, audit logging, and regulatory compliance (APRA, Privacy Act) integration.



2. Technologies & Trade-offs

GCP services, tools, key decisions and challenges

GCP Cloud Services Stack

AI / ML

Vertex AI | AutoML | Vertex AI Search | Gemini API | TensorFlow on TPU

Compute

GKE (Kubernetes) | Cloud Run | Cloud Functions | Compute Engine (GPU)

Data

BigQuery | Cloud SQL (PostgreSQL) | Firestore | Memorystore (Redis) | Bigtable

Storage

Cloud Storage (GCS) | Artifact Registry | Container Registry

Pipeline

Argo Workflow | Pub/Sub | Cloud Composer (Airflow) | Vertex Pipelines (Kubeflow)

DevOps

Cloud Build | Cloud Deploy | Terraform | Cloud Monitoring | Cloud Logging

Security

Secret Manager | Cloud KMS | IAM | VPC Service Controls | Cloud Armor

Key Decisions & Trade-offs

Vertex AI vs. SageMaker

Chose Vertex AI

Native BigQuery integration, Kubeflow pipelines, TPU access for TensorFlow training. Trade-off: Less mature PyTorch ecosystem on GCP at the time.

GKE vs. Cloud Run for Serving

Hybrid approach

GKE for ML model serving (GPU pods, custom scaling). Cloud Run for stateless API microservices. Trade-off: Higher ops cost with GKE but needed GPU scheduling.

LangChain vs. Custom Agents

LangChain + custom

LangChain for agent orchestration, custom tool wrappers for lending domain logic. Trade-off: Framework lock-in vs. speed of development.

Feature Store: Feast vs. Vertex

Vertex AI Feature Store

Managed service, native BQ integration, online/offline serving. Trade-off: Less flexibility than Feast, vendor lock-in.

Challenges Faced & Solutions



Data Quality & Sparsity

Challenge: Incomplete loan applications, missing bureau data, inconsistent formats across origination systems.

Solution: Built robust feature imputation pipeline with MICE + domain heuristics; Great Expectations for schema validation.



Model Fairness & Bias

Challenge: Regulatory requirement (APRA) to ensure no discriminatory lending decisions by protected attributes.

Solution: Implemented fairness-aware ML with Aequitas toolkit; SHAP explainability for every decision; bias audit pipeline.



Real-time vs. Batch Scoring

Challenge: Sub-200ms latency for real-time decisions while supporting batch scoring for portfolio re-assessment.

Solution: Dual serving: Vertex AI Endpoints for real-time (GPU-optimized); Dataflow batch pipeline for portfolio scoring.



Regulatory Compliance

Challenge: Model explainability requirements, audit trails, data sovereignty (AU region only), PII handling.

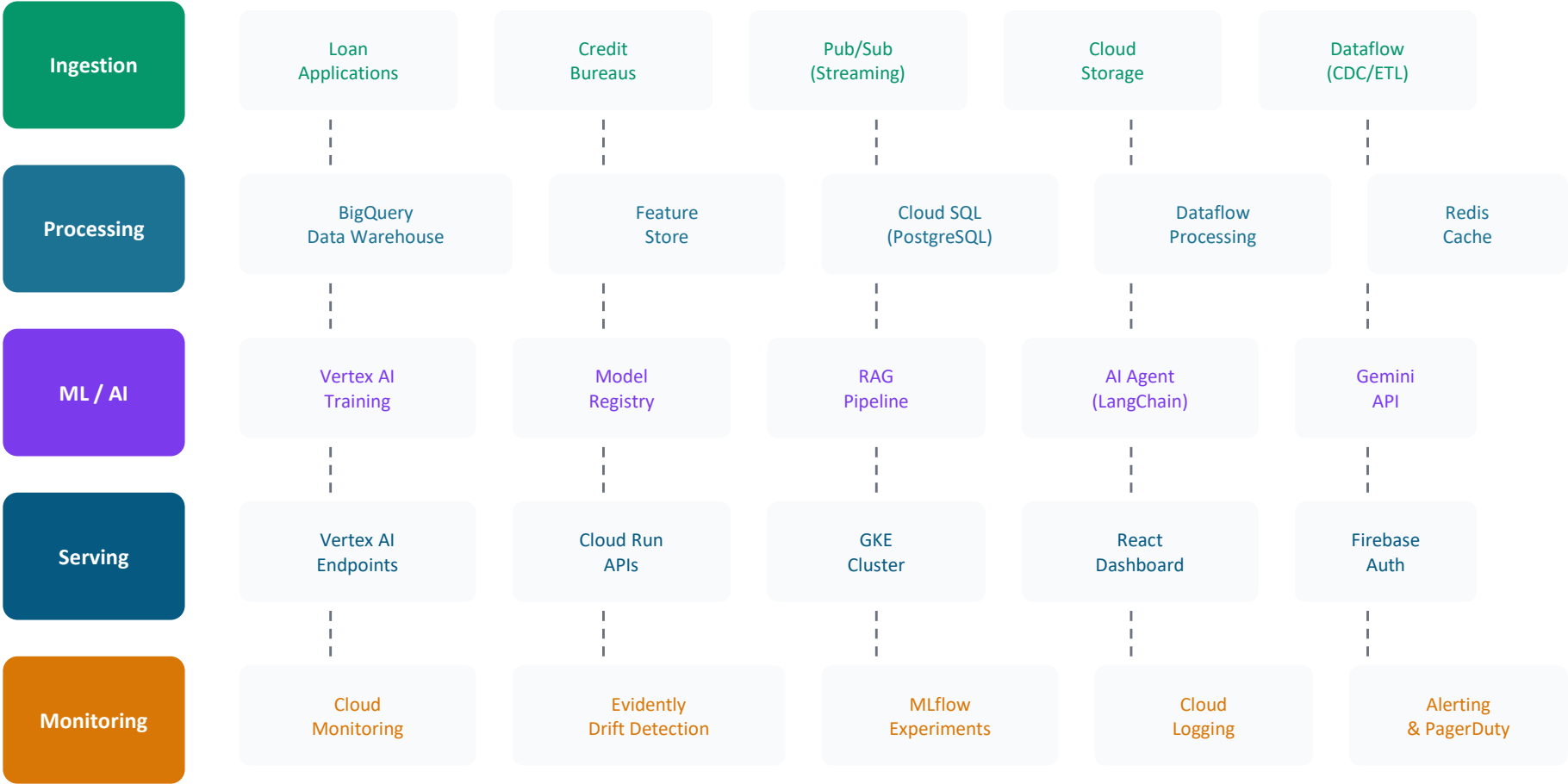
Solution: SHAP-based explanations stored per decision; Cloud Audit Logs; australia-southeast1 region lock; DLP API for PII.



3. Solution Architecture

End-to-end platform architecture on GCP

Solution Architecture — Lending Decision Platform

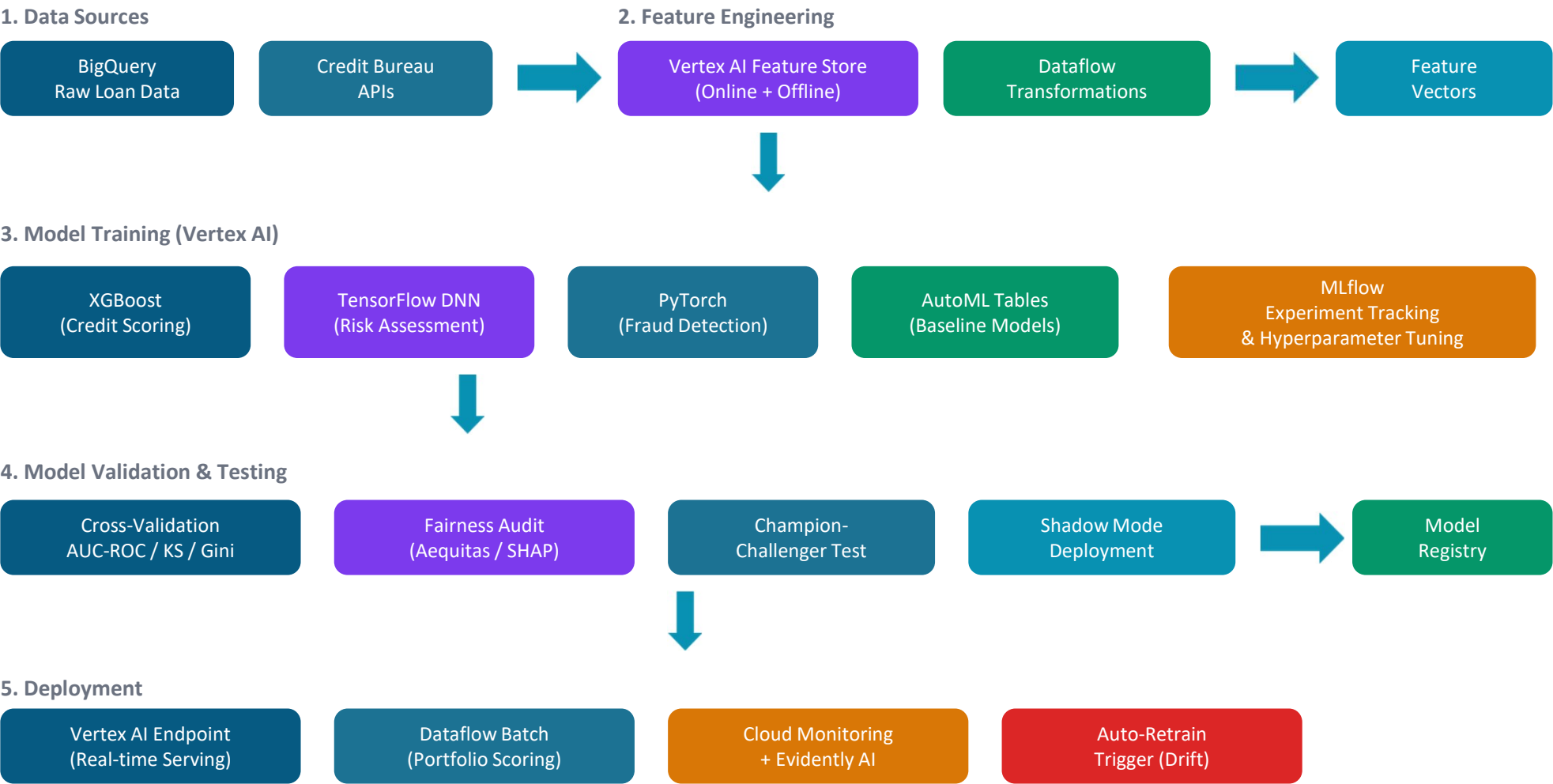




4. ML Model Training Pipeline

How models are trained, validated, and deployed on GCP

ML Model Training Pipeline — Detailed Flow



ML Model Training — Key Details

Credit Scoring Model (XGBoost)

Dataset: 2.5M historical loan records from BigQuery

Features: 180+ engineered features (bureau, income, employment, behavioral)

Target: Binary default (PD) + LGD regression

Training: Vertex AI Custom Training with HPO (Bayesian)

Metrics: AUC-ROC 0.89, KS 0.62, Gini 0.78

Risk Assessment DNN (TensorFlow)

Architecture: 4-layer DNN with dropout + batch norm

Hardware: TPU v3-8 pods on Vertex AI

Features: Tabular + sequential payment history (LSTM branch)

Training: Distributed training with `tf.distribute.Strategy`

Metrics: AUC 0.91, calibration error < 2%

Fraud Detection (PyTorch)

Architecture: Graph Neural Network (GNN) for transaction networks

Framework: PyTorch Geometric on GPU (A100)

Features: Transaction graph + temporal features

Training: Contrastive learning for anomaly detection

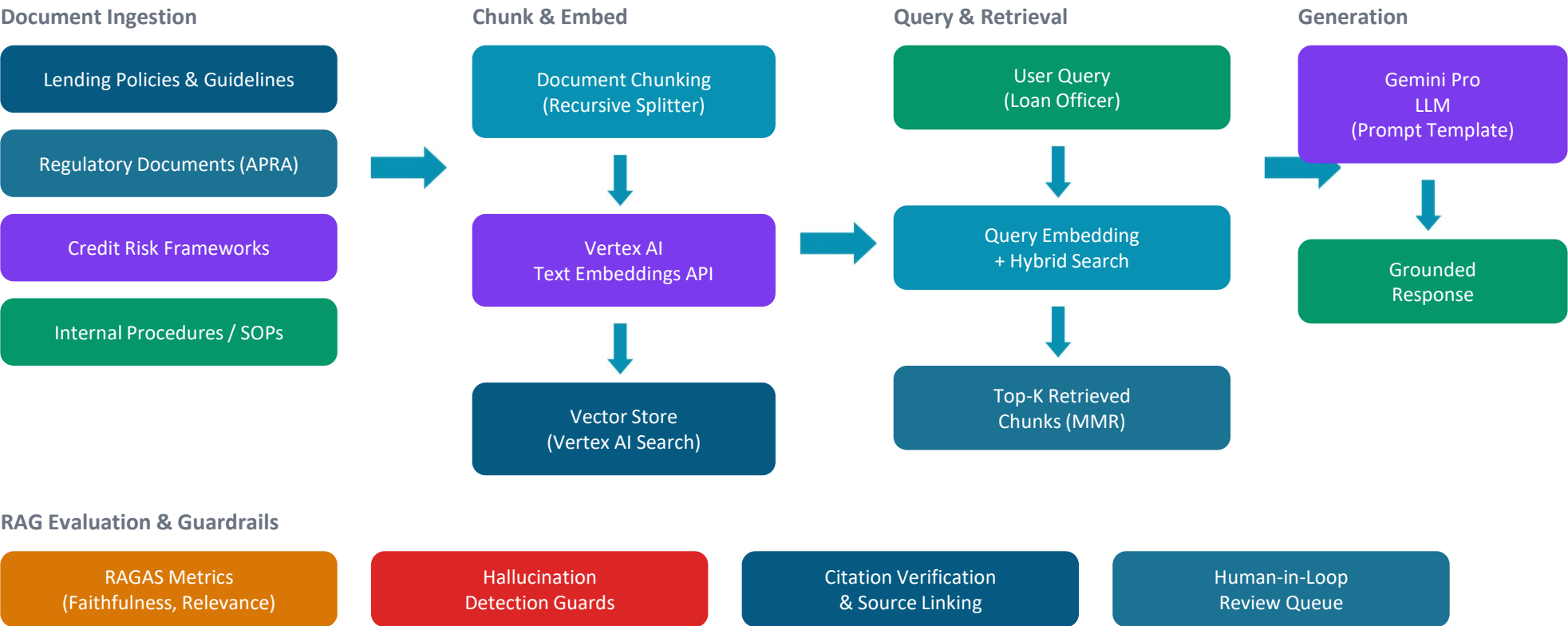
Metrics: Precision@95%Recall = 0.87, F1 = 0.91



5. AI Agent & RAG Pipeline

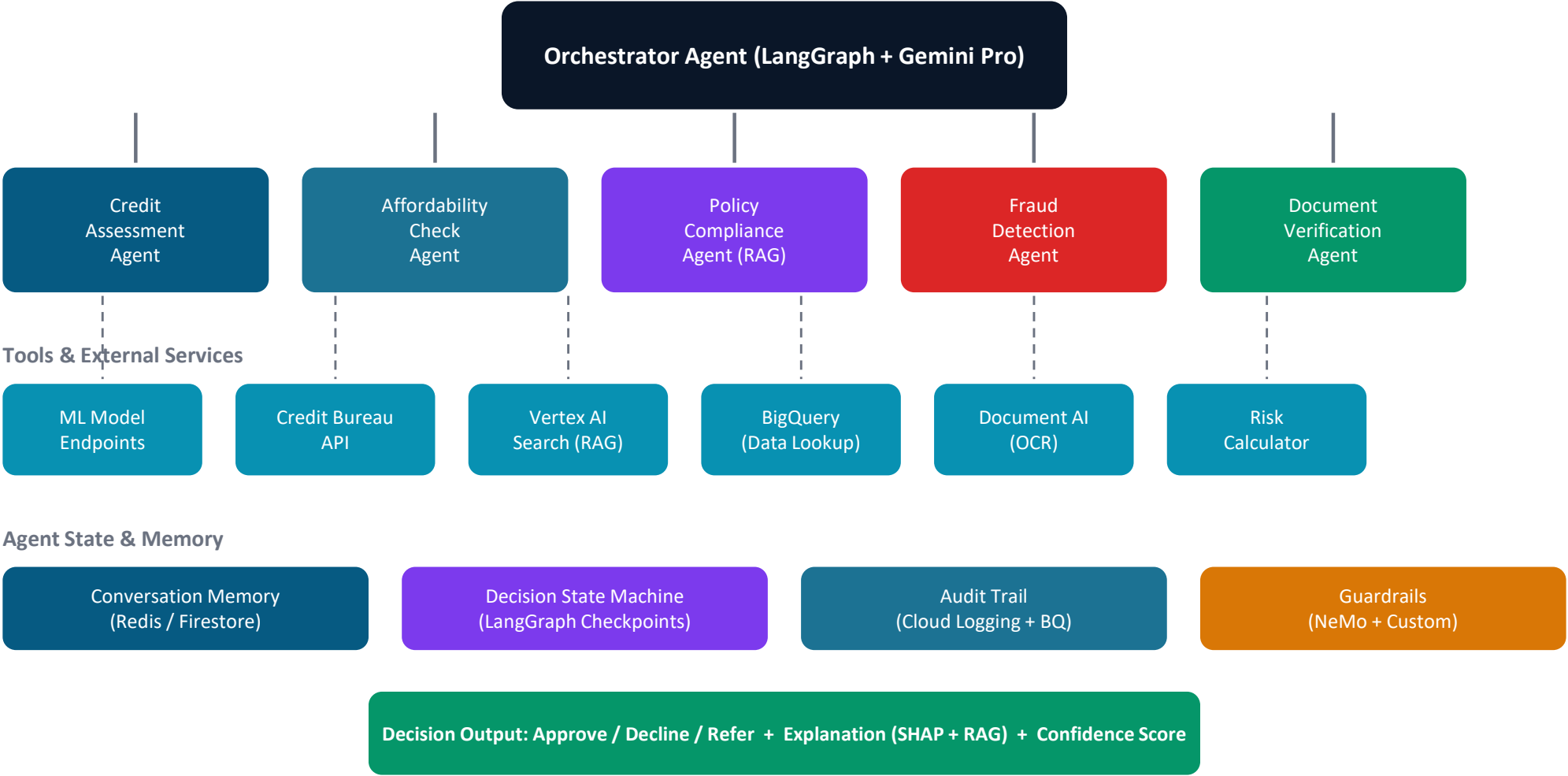
Retrieval-Augmented Generation for lending decisions

RAG Pipeline Architecture



Key Metrics: Retrieval Precision: 92% | Answer Faithfulness: 95% | Context Relevance: 89% | Latency P95: 1.8s | Hallucination Rate: < 3%

AI Agent Architecture — Multi-Agent Lending System

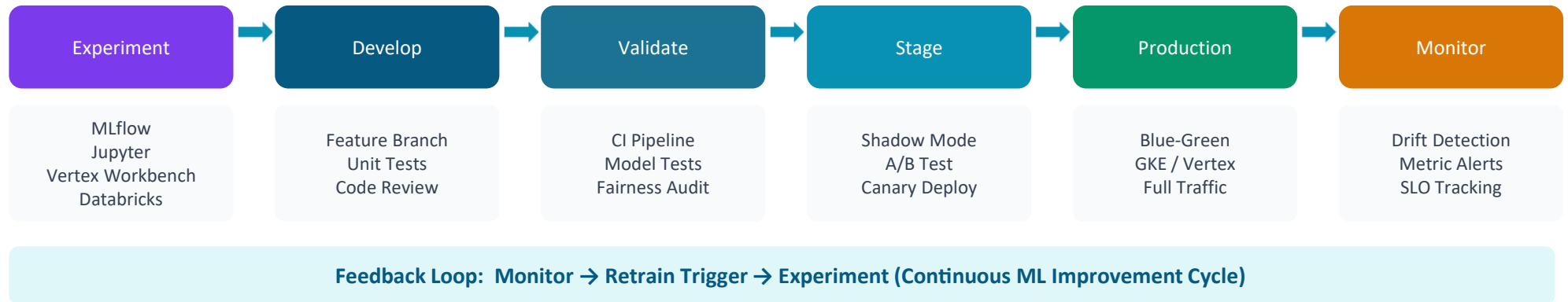




6. MLOps & Release Management

From experiment to production — fast, safe, and measured

Release Management Cycle — ML Model Lifecycle



Key Safeguards for Production

Shadow Deployment

New models run in parallel receiving real traffic but not serving decisions. Compare predictions against champion model for 2-4 weeks.

Blue-Green on GKE

Two identical GKE deployments. Traffic switches via Istio service mesh. Instant rollback by flipping the virtual service.

Canary Releases

Route 5% → 10% → 25% → 50% → 100% of traffic. Automated rollback if KS statistic drops > 5% or latency P99 > 500ms.

Feature Flags

LaunchDarkly integration for model version toggles. Kill-switch per model component. Region-specific rollouts.

ML Framework Strategies & Experiment Workflow

MLflow — Experiment Tracking & Registry

Experiment Tracking: Log params, metrics (AUC, KS, Gini), artifacts per run

Model Registry: Staging → Production promotion with approval gates

Autologging: XGBoost, TF, PyTorch auto-capture via `mlflow.autolog()`

Serving: MLflow models deployed to Vertex AI Endpoints via CI/CD

TensorFlow — Deep Learning Models

Training: Distributed on TPU v3 via `tf.distribute.TPUStrategy`

Architecture: Keras Functional API — DNN + LSTM hybrid for sequential data

Serving: TensorFlow Serving on GKE with GPU autoscaling

Export: SavedModel format → Vertex AI Endpoint + TFLite for edge

PyTorch — Fraud & Graph Models

Training: PyTorch Lightning + DDP on multi-GPU (A100) Vertex AI

Architecture: GNN (PyTorch Geometric) for transaction network fraud

Export: TorchScript → ONNX → Triton Inference Server on GKE

Innovation: Faster R&D iteration — notebook → script → container pipeline

Experiment → Production Workflow

1. Notebook exploration (Vertex Workbench) and Databricks notebook with MLflow
2. Refactor to training script + MLflow logging
3. Containerize → Vertex AI Custom Training Job
4. Register best model in MLflow / Vertex Model Registry
5. CI/CD: Cloud Build → Deploy to Staging → Shadow → Prod
6. Monitor + auto-retrain on drift detection trigger

Model Monitoring & Accuracy Measurement

Real-time Model Monitoring Dashboard (Cloud Monitoring + Splunk + SignalFX + Evidently AI)

AUC-ROC

0.89

Healthy

KS Statistic

0.62

Healthy

PSI (Stability)

0.08

Watch

Gini Coeff

0.78

Healthy

Latency P95

142ms

Healthy

Data Drift Detection

Evidently AI monitors feature distributions hourly. PSI > 0.2 triggers alert. Kolmogorov-Smirnov test for continuous features; Chi-squared for categorical.

Concept Drift Detection

ADWIN algorithm tracks prediction distribution shifts. Ground truth labels joined after 90-day observation window. Model retrain triggered on AUC drop > 3%.

SLO Monitoring

Latency: P95 < 200ms, P99 < 500ms. Availability: 99.9%. Error rate < 0.1%. Monitored via Cloud Monitoring with PagerDuty escalation.



7. Failure Handling & Resilience

Agent hallucination, data/ML drift, and system failures

AI Agent Failure Modes & Mitigations

Agent Hallucination

Agent generates fabricated policy rules, invents credit criteria, or provides inaccurate regulatory guidance that could lead to non-compliant lending decisions.

- 1) RAG grounding — every response must cite retrieved chunks with source IDs
- 2) Confidence thresholds — responses below 0.7 confidence routed to human review
- 3) NeMo Guardrails — topical rails prevent off-topic generation; fact-checking rail validates against known policies
- 4) Output validation — structured output schema enforced via Pydantic; regex checks for numerical claims

Bad Tool Arguments / Wrong Tool Selection

Agent passes malformed arguments to credit bureau API (wrong ID format), selects affordability tool when fraud check was needed, or chains tools in wrong order.

- 1) Tool schemas with Pydantic validation — reject malformed args before execution
- 2) ReAct prompting with few-shot examples for each tool
- 3) Tool selection guardrails — decision tree pre-filter before LLM tool choice
- 4) Retry with reflection — on tool error, agent receives error + self-corrects (max 3 retries)

Context Overload / Window Exhaustion

Complex loan applications with 20+ documents exceed context window. Agent loses track of earlier information, contradicts itself, or drops critical data.

- 1) Hierarchical summarization — pre-summarize each document, load full text only on-demand
- 2) Sliding window with memory — key facts extracted to structured state (LangGraph)
- 3) Multi-turn decomposition — break complex assessments into sub-tasks with checkpoints
- 4) Token budget management — monitor usage, trigger summarization at 80% capacity

Data/ML Failures & System Failures

Data / ML Failures

Quality Drift

Upstream schema changes, missing fields, new categorical values. Mitigation: Great Expectations data contracts with automated alerts. Feature store validates schema on write. Fallback to last-known-good feature snapshot.

Concept Drift

Economic shifts (e.g., rate changes) alter default patterns. Mitigation: ADWIN detector on prediction distributions. Retrain pipeline auto-triggered when PSI > 0.2. Champion-challenger evaluation before promotion.

Missing Feature Values

Bureau data unavailable, employment API timeout. Mitigation: Feature importance ranking — if top-5 feature missing, route to manual. Imputation with MICE for non-critical features. Graceful degradation to simpler model.

System / Tool Failures

API Timeouts

Credit bureau or external API exceeds SLA. Mitigation: Circuit breaker pattern (Istio). Exponential backoff with jitter (max 3 retries). Cached last-known response with staleness flag. Fallback to async processing with callback.

Schema Mismatches

External API changes payload structure without notice. Mitigation: Contract testing (Pact) in CI/CD. Schema registry for internal events (Pub/Sub + Protobuf). Versioned API adapters with backward compatibility layer.

Multi-Agent Orchestration Bottlenecks

Agent deadlocks, circular tool calls, cascading failures. Mitigation: Timeout per agent step (30s default). LangGraph state machine prevents cycles. Dead letter queue for failed orchestrations. Async fan-out with Pub/Sub for independent sub-agents.



Key Takeaways



End-to-end ML platform on GCP: from data ingestion to real-time lending decisions in < 200ms



Multi-model architecture: XGBoost + TensorFlow + PyTorch serving different lending domain needs



RAG-powered AI agents: grounded, explainable decisions with regulatory compliance built-in



MLOps maturity: automated retraining, drift detection, shadow deployment, and canary releases



Resilience-first design: circuit breakers, graceful degradation, and comprehensive guardrails

Thank You

